

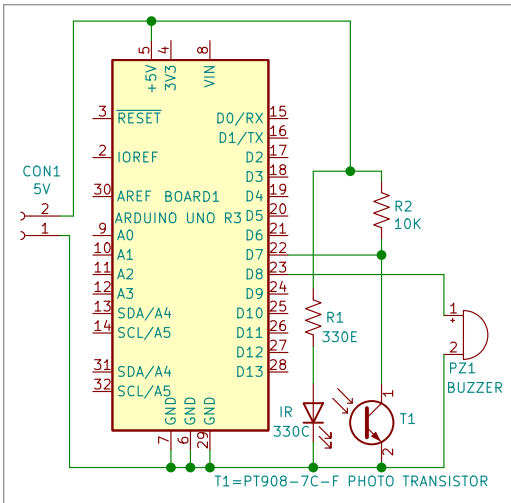


Fig. 4: Circuit diagram of an IR module with Arduino Uno

```
#define SOUNDER_PIN 8
#define TRIGGER_PIN 7

int FrequencyArr[4] = {350, 550, 750, 950};
int Indx = 0; // Index variable

void setup() {
  pinMode(SOUNDER_PIN, OUTPUT);
  pinMode(TRIGGER_PIN, INPUT_PULLUP);
}

void loop() {
  if (digitalRead(TRIGGER_PIN) == HIGH) {
    noTone(SOUNDER_PIN);
  } else {
    tone(SOUNDER_PIN, FrequencyArr[Indx]);
    delay(500);
    Indx++;
    if (Indx == 4) {
      Indx = 0;
    }
  }
}
```

Fig. 5: Snippet of the source code

Arduino Uno. The device is built using an Arduino Uno (Board1), an infrared (IR) transmitting LED, a phototransistor, and a few additional components. Instead of a ready-made infrared sensor module, a single infrared LED (IR330C), a phototransistor (PT908-7C-F), and discrete components are used.

The setup may appear excessive, but it does not involve significant additional cost. Many electronics enthusiasts are likely already to have an Arduino Uno or Nano board available. The Arduino Uno circuit, as shown in Fig. 4, is straightforward and self-explanatory.

The working principle is simple: the buzzer produces tones of varying frequencies depending on the amount of light received by the phototransistor after it reflects off an object.

Software

The sketch (software) is designed such that every time the infrared reflectance sensor triggers the system, it plays a different frequency tone (up to four tones, as defined in the code). When the frequency array reaches the last index, it automatically resets to the beginning on the next trigger. Fig. 5 shows a snippet of the source code.

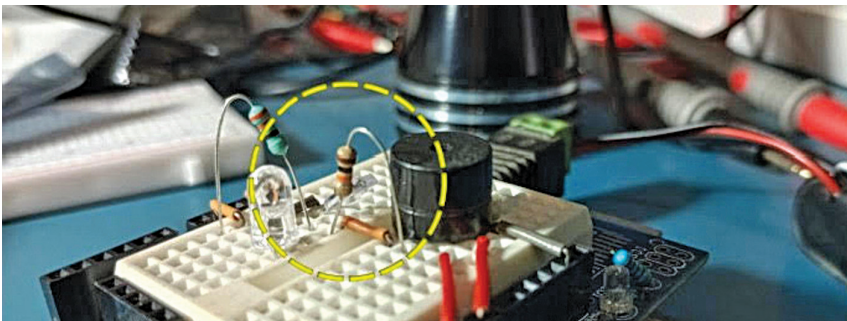


Fig. 6: Author's prototype

TO DOWNLOAD SOURCE CODE AND RESOURCES



Scan this QR code or go to <https://sourcecode.electronicsforu.com> to access and download source code and other resources for this project

Construction and testing

To test the device, upload the source code to the Arduino Uno after downloading. Then assemble the circuit on a breadboard, as shown in the prototype in Fig. 6. After that, functional testing can be performed.

The optimal sensing range of the prototype is approximately 10cm. Adding an external 10kΩ pull-up resistor to the phototransistor (between Uno D7 and the +5V rail) can significantly reduce the detection range to less than 1cm. This resistor maintains a defined logic high on the digital pin when it is not actively pulled low by the phototransistor.

The reflectance sensor mechanism is basic, as the infrared light is not modulated and relies solely on a light threshold. Significant changes in ambient light can affect performance. To mitigate this, it is recommended to create a shield around the sensor area to reduce exposure to ambient light and adjust the sensor's orientation to minimise interference further.

This Arduino IR reflectance sensor can be easily modified and adapted for various practical applications, including line-following robots and proximity or edge detection. **EFY**



T.K. Hareendran is an electronics designer, hardware beta tester, technical author, and product reviewer